

Guía Completa de Consultas SQL

MySQL — Basada en el Tema 4: Realización de consultas

SELECT · WHERE · JOIN · GROUP BY · HAVING · Subconsultas · Vistas · Tablas derivadas

Incluye explicación de cada palabra reservada del temario, ejemplos prácticos y ejercicios tipo examen con soluciones.

Bases de Datos — 1º DAM — Curso 2025/2026

Índice de contenidos

1. Sentencia SELECT básica
2. Palabras reservadas del temario
3. Funciones SQL
 - 3.1 Funciones lógicas (IFNULL, IF)
 - 3.2 Funciones numéricas (COUNT, ROUND, SUM, MIN, MAX, AVG)
 - 3.3 Funciones de texto (CONCAT, UPPER, LOWER, LTRIM, RTRIM)
 - 3.4 Funciones de fecha (NOW, CURDATE, DATEDIFF, TIMEDIFF, YEAR, MONTH, DATE_FORMAT)
4. Filtros (WHERE)
 - 4.1 Operadores de comparación
 - 4.2 IN — pertenencia a conjunto
 - 4.3 BETWEEN — rango
 - 4.4 IS NULL / IS NOT NULL
 - 4.5 LIKE — patrones
 - 4.6 LIMIT
5. Ordenación (ORDER BY)
6. Consultas resumen (agregación)
 - 6.1 GROUP BY
 - 6.2 HAVING
7. Subconsultas
8. Consultas multitable (JOINS)
 - 8.1 INNER JOIN
 - 8.2 LEFT / RIGHT JOIN (OUTER JOIN)
 - 8.3 FULL OUTER JOIN
 - 8.4 CROSS JOIN / NATURAL JOIN
 - 8.5 Consultas reflexivas
9. Operadores de conjuntos (UNION, INTERSECT, EXCEPT)
10. Tablas derivadas
11. Vistas (CREATE VIEW)
12. Resumen de palabras reservadas

1. Sentencia SELECT básica

La sentencia **SELECT** es la instrucción fundamental de SQL para consultar datos de las tablas. Es la sentencia más versátil y compleja de todo el lenguaje SQL. Su forma básica es:

```
SELECT columna1, columna2, ...  
FROM nombre_tabla;
```

Para seleccionar **todas** las columnas se usa el asterisco *****:

```
SELECT *  
FROM nombre_tabla;
```

Se puede dar un **alias** a cada columna con la palabra **AS**. Esto es muy recomendable cuando la consulta se realiza desde un programa (por ejemplo Java) para aislar el código de la estructura de la base de datos:

```
SELECT nombre AS empleado, salario AS sueldo  
FROM empleados;
```

Para eliminar filas duplicadas se usa **DISTINCT**:

```
SELECT DISTINCT departamento  
FROM empleados;
```

También se pueden usar **expresiones calculadas** en el SELECT:

```
SELECT nombre, salario, salario * 12 AS salario_anual  
FROM empleados;
```

2. Palabras reservadas del temario

Estas son las palabras clave que forman la estructura de las consultas SQL y que aparecen en el temario. Cada una tiene un propósito específico:

SELECT — Indica qué columnas o expresiones quieres obtener.

```
SELECT nombre, salario FROM empleados;
```

FROM — Especifica la tabla (o tablas) de donde se leen los datos.

```
SELECT * FROM empleados;
```

AS — Crea un alias (nombre alternativo) para columnas o tablas.

```
SELECT nombre AS empleado FROM empleados;
```

DISTINCT — Elimina filas duplicadas del resultado.

```
SELECT DISTINCT ciudad FROM empleados;
```

WHERE — Filtra filas según una condición. Solo devuelve las que la cumplen.

```
SELECT * FROM empleados WHERE salario > 3000;
```

AND / OR / NOT — Combinan condiciones lógicas dentro de WHERE.

```
SELECT * FROM empleados  
WHERE departamento = 'IT' AND salario > 3000;
```

IN — Comprueba si un valor pertenece a un conjunto.

```
SELECT * FROM empleados
WHERE ciudad IN ('Madrid', 'Barcelona');
```

BETWEEN — Comprueba si un valor está dentro de un rango (inclusivo).

```
SELECT * FROM empleados
WHERE salario BETWEEN 2500 AND 3500;
```

LIKE — Busca patrones en texto. % = cualquier cadena, _ = un carácter.

```
SELECT * FROM empleados
WHERE nombre LIKE 'A%';
```

IS NULL / IS NOT NULL — Comprueba si un valor es nulo o no.

```
SELECT * FROM empleados
WHERE departamento IS NOT NULL;
```

ORDER BY — Ordena los resultados. ASC = ascendente (por defecto), DESC = descendente.

```
SELECT * FROM empleados
ORDER BY salario DESC;
```

LIMIT — Limita el número de filas devueltas.

```
SELECT * FROM empleados
ORDER BY salario DESC LIMIT 3;
```

GROUP BY — Agrupa filas que comparten un valor para aplicar funciones de agregación.

```
SELECT departamento, AVG(salario)
FROM empleados GROUP BY departamento;
```

HAVING — Filtra grupos (como WHERE pero para resultados agrupados).

```
SELECT departamento, AVG(salario) AS media
FROM empleados
GROUP BY departamento
HAVING media > 3000;
```

JOIN / INNER JOIN — Combina filas de dos tablas donde hay coincidencia.

```
SELECT e.nombre, d.nombre_dep
FROM empleados e
INNER JOIN departamentos d
ON e.id_dep = d.id_dep;
```

LEFT JOIN — Devuelve todas las filas de la tabla izquierda, con NULL si no hay coincidencia.

```
SELECT e.nombre, d.nombre_dep
FROM empleados e
LEFT JOIN departamentos d
ON e.id_dep = d.id_dep;
```

RIGHT JOIN — Igual que LEFT JOIN pero al revés: todas las filas de la tabla derecha.

```
SELECT e.nombre, d.nombre_dep
FROM empleados e
RIGHT JOIN departamentos d
ON e.id_dep = d.id_dep;
```

FULL OUTER JOIN — Devuelve todas las filas de ambas tablas, con NULL donde no haya coincidencia.

```
-- MySQL no lo soporta, se simula con UNION:  
SELECT ... FROM a LEFT JOIN b ON ...  
UNION  
SELECT ... FROM a RIGHT JOIN b ON ...;
```

CROSS JOIN — Producto cartesiano: cada fila de A con cada fila de B.

```
SELECT * FROM colores CROSS JOIN tallas;
```

NATURAL JOIN — JOIN automático por columnas con el mismo nombre.

```
SELECT * FROM empleados NATURAL JOIN departamentos;
```

ON — Especifica la condición de unión en un JOIN.

```
... INNER JOIN departamentos d ON e.id_dep = d.id_dep;
```

UNION — Combina resultados de dos consultas eliminando duplicados.

```
SELECT ciudad FROM clientes  
UNION  
SELECT ciudad FROM proveedores;
```

INTERSECT — Devuelve filas comunes a ambas consultas.

```
SELECT ciudad FROM clientes  
INTERSECT  
SELECT ciudad FROM proveedores;
```

EXCEPT (MINUS) — Filas de la primera consulta que NO están en la segunda.

```
SELECT ciudad FROM clientes  
EXCEPT  
SELECT ciudad FROM proveedores;
```

EXISTS / NOT EXISTS — TRUE si la subconsulta produce algún resultado / ninguno.

```
SELECT nombre FROM clientes c  
WHERE EXISTS (  
  SELECT 1 FROM pedidos p  
  WHERE p.id_cliente = c.id_cliente  
);
```

ALL / ANY — Comparan contra todos (ALL) o alguno (ANY) de una subconsulta.

```
SELECT * FROM empleados  
WHERE salario > ALL (  
  SELECT salario FROM empleados  
  WHERE departamento = 'Ventas'  
);
```

CREATE VIEW — Crea una tabla virtual basada en una consulta.

```
CREATE VIEW vista_it AS  
SELECT nombre, salario  
FROM empleados  
WHERE departamento = 'IT';
```

DROP VIEW — Elimina una vista existente.

```
DROP VIEW vista_it;
```

3. Funciones SQL

Es común utilizar funciones ya predefinidas a la hora de consultar y procesar datos, ya que facilitan el trabajo evitando tener que procesar los datos obtenidos desde el programa.

3.1 Funciones lógicas

IFNULL(expr, valor_si_nulo) — Si expr es NULL, devuelve valor_si_nulo. Si no es NULL, devuelve expr.

```
SELECT IFNULL(NULL, 'valor nulo') AS valor;
+-----+
| valor |
+-----+
| valor nulo |
+-----+
```

IF(expr, valor_si_true, valor_si_false) — Si expr es true devuelve valor_si_true, si no devuelve valor_si_false.

```
SELECT IF(5>5, 'verdadero', 'falso') AS condicion;
+-----+
| condicion |
+-----+
| falso |
+-----+
```

3.2 Funciones numéricas

COUNT(*) — Devuelve el total de registros recuperados.

```
SELECT COUNT(*) FROM empleados;

-- Se puede combinar con DISTINCT:
SELECT COUNT(DISTINCT departamento) AS total_dep
FROM empleados;
```

ROUND(num, decimales) — Redondea el número a los decimales especificados.

```
SELECT ROUND(1234.499, 2);
-- Resultado: 1234.50
```

SUM(X) — Devuelve la suma de los valores.

```
SELECT SUM(salario) AS total_salarios FROM empleados;
-- Resultado: 20400.00
```

MIN(X) / MAX(X) — Devuelve el valor mínimo / máximo.

```
SELECT MIN(salario) AS salario_minimo FROM empleados;
-- Resultado: 2400.00

SELECT MAX(salario) AS salario_maximo FROM empleados;
-- Resultado: 3500.00
```

AVG(X) — Devuelve la media aritmética.

```
SELECT AVG(salario) AS salario_medio FROM empleados;
-- Resultado: 2914.285714
```

3.3 Funciones de texto

CONCAT(X, Y, ...) — Concatena n cadenas de texto.

```
SELECT CONCAT(nombre, '-', departamento) AS nombre_dep
FROM empleados;
-- Resultado: 'Ana García-Ventas', 'Luis López-IT', ...
```

UPPER(X) / LOWER(X) — Transforma a mayúsculas / minúsculas.

```
SELECT UPPER(nombre) FROM empleados;
-- ANA GARCÍA, LUIS LÓPEZ, ...
```

```
SELECT LOWER(nombre) FROM empleados;
-- ana garcía, luis lópez, ...
```

LTRIM(X) / RTRIM(X) — Elimina espacios por delante (LTRIM) / por detrás (RTRIM).

```
SELECT LTRIM(' hola') AS sin_espacios_izq;
-- Resultado: 'hola'
```

```
SELECT RTRIM('hola ') AS sin_espacios_der;
-- Resultado: 'hola'
```

3.4 Funciones de fecha

NOW() / CURRENT_TIMESTAMP() — Devuelve la fecha y hora actuales.

```
SELECT CURRENT_TIMESTAMP(), NOW();
-- 2026-01-08 13:24:51
```

CURDATE() — Devuelve la fecha actual (sin hora).

```
SELECT CURDATE();
-- 2026-01-08
```

```
SELECT CURDATE() + 1;
-- 20260109
```

DATEDIFF(fecha1, fecha2) — Diferencia en días entre dos fechas.

```
SELECT DATEDIFF('2026-01-25 09:34:21', '2026-01-15 15:25:35');
-- Resultado: 10
```

TIMEDIFF(hora1, hora2) — Diferencia entre dos horas.

```
SELECT TIMEDIFF('13:10:11', '14:10:10');
-- Resultado: -00:59:59
```

YEAR(X) / MONTH(X) — Devuelve el año (yyyy) / mes (mm) de una fecha.

```
SELECT MONTH(fecha_ingreso) AS mes_ingreso,
YEAR(fecha_ingreso) AS anio_ingreso
FROM empleados;
```

DATE_FORMAT(fecha_hora, formato) — Devuelve una fecha formateada según el patrón indicado.

```
SELECT DATE_FORMAT(NOW(), '%e/%m/%Y %H:%i:%s') AS instante;  
-- Resultado: 3/01/2026 09:03:41
```

Principales códigos de formato para DATE_FORMAT:

Código	Descripción	Ejemplo
%Y	Año 4 dígitos	2026
%y	Año 2 dígitos	26
%m	Mes numérico (01-12)	03
%M	Nombre del mes completo	January
%d	Día del mes (01-31)	08
%e	Día del mes (0-31)	8
%H	Hora 24h (00-23)	13
%h	Hora 12h (00-12)	01
%i	Minutos (00-59)	24
%s	Segundos (00-59)	51
%W	Nombre del día completo	Wednesday
%a	Nombre del día abreviado	Wed
%p	AM o PM	PM
%T	Hora en formato 24h (hh:mm:ss)	13:24:51

4. Filtros (WHERE)

La cláusula **WHERE** filtra las filas que se devuelven. Solo se incluyen aquellas filas que cumplen la condición especificada.

4.1 Operadores de comparación

Los operadores básicos son: = (igual), != o <> (distinto), < (menor), > (mayor), <= (menor o igual), >= (mayor o igual). Se pueden combinar con **AND**, **OR** y **NOT**.

```
-- Empleados con salario mayor a 3000
SELECT * FROM empleados WHERE salario > 3000;

-- Combinar con AND
SELECT * FROM empleados
WHERE departamento = 'IT' AND salario > 3000;

-- Combinar con OR
SELECT * FROM empleados
WHERE ciudad = 'Madrid' OR ciudad = 'Barcelona';
```

4.2 IN — pertenencia a conjunto

Comprueba si un valor está dentro de una lista. Equivale a múltiples OR.

```
SELECT * FROM empleados
WHERE ciudad IN ('Madrid', 'Barcelona', 'Valencia');

-- Equivalente a:
SELECT * FROM empleados
WHERE ciudad = 'Madrid' OR ciudad = 'Barcelona'
OR ciudad = 'Valencia';
```

4.3 BETWEEN — rango

Filtra valores dentro de un rango **inclusivo** (incluye ambos extremos). Equivale a usar >= y <=.

```
SELECT * FROM empleados
WHERE salario BETWEEN 2500 AND 3200;

-- Equivalente a:
SELECT * FROM empleados
WHERE salario >= 2500 AND salario <= 3200;
```

4.4 IS NULL / IS NOT NULL

NULL representa la **ausencia de valor**. No se puede comparar con = o !=. Se debe usar IS NULL o IS NOT NULL.

```
-- Empleados sin departamento asignado
SELECT * FROM empleados WHERE departamento IS NULL;

-- Empleados con departamento asignado
```

```
SELECT * FROM empleados WHERE departamento IS NOT NULL;
```

4.5 LIKE — patrones

Se utiliza para seleccionar registros en función de comodines con cadenas de texto. Los comodines son: % (cualquier secuencia de caracteres, incluida vacía) y _ (exactamente un carácter).

```
-- Nombres que empiezan por 'A'
SELECT * FROM empleados WHERE nombre LIKE 'A%';

-- Nombres que terminan en 'a'
SELECT * FROM empleados WHERE nombre LIKE '%a';

-- Nombres que contienen 'ar'
SELECT * FROM empleados WHERE nombre LIKE '%ar%';

-- Segunda letra es 'u'
SELECT * FROM empleados WHERE nombre LIKE '_u%';
```

4.6 LIMIT

Restringe el número de filas devueltas. Muy útil combinado con ORDER BY.

```
-- Los 3 empleados mejor pagados
SELECT * FROM empleados
ORDER BY salario DESC
LIMIT 3;
```

5. Ordenación (ORDER BY)

Ordena los resultados por una o más columnas. Por defecto es **ASC** (ascendente). Usa **DESC** para orden descendente.

```
-- Ordenar por salario de mayor a menor
SELECT * FROM empleados ORDER BY salario DESC;

-- Ordenar por departamento (A-Z) y dentro por salario (mayor primero)
SELECT * FROM empleados
ORDER BY departamento ASC, salario DESC;

-- Ordenar por posición numérica de columna
SELECT nombre, departamento, salario
FROM empleados
ORDER BY 3 DESC; -- 3 = tercera columna = salario
```

6. Consultas resumen (agregación)

Las consultas resumen/agrupadas se utilizan para obtener información resumida, por ejemplo la suma total de un conjunto de registros, la media aritmética, etc. Siempre aparecerá alguna de estas funciones: **SUM**, **AVG**, **MIN**, **MAX**, **COUNT**.

```
SELECT SUM(salario) AS total_salarios FROM empleados;

SELECT AVG(salario) AS salario_medio FROM empleados;

SELECT MAX(salario) AS maximo, MIN(salario) AS minimo
FROM empleados;

SELECT COUNT(*) AS total_empleados FROM empleados;

-- COUNT(columna) no cuenta los nulos
-- COUNT(*) cuenta todos los registros incluidos los nulos
```

6.1 GROUP BY — agrupación

Agrupar las filas que comparten el mismo valor en una columna. Las funciones de agregación se aplican **dentro de cada grupo**.

```
-- Salario medio por departamento
SELECT departamento, AVG(salario) AS salario_medio
FROM empleados
GROUP BY departamento;

-- Número de empleados por ciudad
SELECT ciudad, COUNT(*) AS total
FROM empleados
GROUP BY ciudad;
```

Nota: Regla importante: toda columna que aparezca en SELECT y NO sea una función de agregación, DEBE aparecer también en GROUP BY.

6.2 HAVING — filtro de grupos

HAVING filtra **después de agrupar**. Es como WHERE pero para grupos. WHERE filtra filas individuales antes de agrupar, HAVING filtra los grupos después.

```
-- Departamentos con salario medio superior a 3000
SELECT departamento, AVG(salario) AS media
FROM empleados
GROUP BY departamento
HAVING media > 3000;

-- Ciudades con más de 2 empleados
SELECT ciudad, COUNT(*) AS total
FROM empleados
GROUP BY ciudad
HAVING total > 2;
```

7. Subconsultas

Las subconsultas se utilizan para realizar filtrados con los datos de otra consulta. Se puede aplicar el filtro tanto en la cláusula WHERE como en HAVING. Es un **SELECT dentro de otro SELECT**.

Subconsulta con comparación (devuelve un solo valor)

```
-- Empleados que ganan más que la media
SELECT nombre, salario
FROM empleados
WHERE salario > (SELECT AVG(salario) FROM empleados);
```

Subconsulta con IN (devuelve un conjunto)

```
-- Empleados de departamentos con presupuesto > 30000
SELECT nombre
FROM empleados
WHERE id_departamento IN (
  SELECT id_departamento
  FROM departamentos
  WHERE presupuesto > 30000
);
```

Subconsulta con EXISTS / NOT EXISTS

EXISTS devuelve TRUE si la subconsulta produce **al menos una fila**. NOT EXISTS devuelve TRUE si la subconsulta NO produce ninguna fila.

```
-- Clientes que tienen al menos un pedido
SELECT nombre FROM clientes c
WHERE EXISTS (
  SELECT 1 FROM pedidos p
  WHERE p.id_cliente = c.id_cliente
);

-- Clientes que NO tienen pedidos
SELECT nombre FROM clientes c
WHERE NOT EXISTS (
  SELECT 1 FROM pedidos p
  WHERE p.id_cliente = c.id_cliente
);
```

Subconsulta con ALL y ANY

ALL: la condición debe cumplirse para TODOS los valores de la subconsulta.

ANY: basta con que se cumpla para AL MENOS UN valor.

```
-- Empleados que ganan más que TODOS los de Ventas
SELECT nombre, salario FROM empleados
WHERE salario > ALL (
  SELECT salario FROM empleados
  WHERE departamento = 'Ventas'
);

-- Empleados que ganan más que ALGUNO de IT
SELECT nombre, salario FROM empleados
```

```
WHERE salario > ANY (  
  SELECT salario FROM empleados  
  WHERE departamento = 'IT'  
);
```

8. Consultas multitabla (JOINS)

Los JOINS permiten **combinar filas de dos o más tablas** relacionadas entre sí. Son esenciales cuando los datos están repartidos en varias tablas.

Nota: Si combinas dos tablas sin condición de JOIN, obtienes un **producto cartesiano**: cada fila de A se combina con cada fila de B.

8.1 INNER JOIN

Devuelve solo las filas que tienen **correspondencia en ambas tablas**.

```
SELECT e.nombre, e.salario, d.nombre_dep
FROM empleados e
INNER JOIN departamentos d ON e.id_dep = d.id_dep;

-- Sintaxis alternativa (sin JOIN, usando WHERE):
SELECT e.nombre, e.salario, d.nombre_dep
FROM empleados e, departamentos d
WHERE e.id_dep = d.id_dep;
```

8.2 LEFT JOIN / RIGHT JOIN (OUTER JOIN)

LEFT JOIN: devuelve TODAS las filas de la tabla **izquierda**. Si no hay correspondencia en la derecha, rellena con NULL.

RIGHT JOIN: al revés, todas las filas de la tabla **derecha**.

```
-- TODOS los empleados, aunque no tengan departamento
SELECT e.nombre, d.nombre_dep
FROM empleados e
LEFT JOIN departamentos d ON e.id_dep = d.id_dep;

-- TODOS los departamentos, aunque no tengan empleados
SELECT e.nombre, d.nombre_dep
FROM empleados e
RIGHT JOIN departamentos d ON e.id_dep = d.id_dep;
```

8.3 FULL OUTER JOIN

Devuelve TODAS las filas de ambas tablas. MySQL no lo soporta directamente, pero se simula con UNION:

```
SELECT e.nombre, d.nombre_dep
FROM empleados e
LEFT JOIN departamentos d ON e.id_dep = d.id_dep

UNION

SELECT e.nombre, d.nombre_dep
FROM empleados e
RIGHT JOIN departamentos d ON e.id_dep = d.id_dep;
```

8.4 CROSS JOIN / NATURAL JOIN

CROSS JOIN: producto cartesiano explícito (todas las combinaciones).

NATURAL JOIN: JOIN automático por las columnas con el mismo nombre en ambas tablas.

```
-- CROSS JOIN
SELECT * FROM colores CROSS JOIN tallas;

-- NATURAL JOIN
SELECT * FROM empleados NATURAL JOIN departamentos;
```

8.5 Consultas reflexivas (Self JOIN)

Una tabla se une **consigo misma**. Se usa cuando una tabla tiene una referencia a sí misma (ej: empleados y sus jefes en la misma tabla). Es imprescindible usar alias distintos para cada "copia" de la tabla.

```
-- Empleados y el nombre de su jefe
SELECT e.nombre AS empleado, j.nombre AS jefe
FROM empleados e
LEFT JOIN empleados j ON e.id_jefe = j.id_empleado;
```

9. Operadores de conjuntos (UNION, INTERSECT, EXCEPT)

Estos operadores combinan los resultados de dos consultas SELECT. Requisito: ambas consultas deben tener el **mismo número de columnas y tipos compatibles**.

UNION — Combina resultados eliminando duplicados.

```
SELECT ciudad FROM clientes
UNION
SELECT ciudad FROM proveedores;
```

INTERSECT — Solo las filas que aparecen en AMBAS consultas.

```
SELECT ciudad FROM clientes
INTERSECT
SELECT ciudad FROM proveedores;
```

EXCEPT (MINUS) — Filas que están en la primera pero NO en la segunda.

```
SELECT ciudad FROM clientes
EXCEPT
SELECT ciudad FROM proveedores;
```

10. Tablas derivadas

Las consultas con tablas derivadas son aquellas que utilizan una consulta SELECT en la cláusula FROM en lugar del nombre de una tabla. Es decir, una consulta de datos que tiene como origen una tabla dinámica/temporal. La subconsulta **DEBE tener un alias**.

```
SELECT ...
FROM
(SELECT ...
FROM ...
) AS tabla_derivada;
```

Ejemplo práctico:

```
-- Clientes cuyo total de pedidos supera la media
SELECT td.nombre, td.total_pedidos
FROM (
SELECT c.nombre, SUM(p.total) AS total_pedidos
FROM clientes c
JOIN pedidos p ON c.id_cliente = p.id_cliente
GROUP BY c.nombre
) AS td
WHERE td.total_pedidos > (
SELECT AVG(total) FROM pedidos
);
```

11. Vistas (CREATE VIEW)

Una vista es una consulta de datos que se comporta como una **tabla virtual**. La diferencia con las tablas derivadas es que la vista **existe en la base de datos** y se comporta como otra tabla más. Se puede reutilizar en cualquier consulta.

Crear una vista

```
CREATE VIEW vista_empleados_it AS
SELECT nombre, salario, fecha_ingreso
FROM empleados
WHERE departamento = 'IT';

-- Ahora se puede consultar como una tabla
SELECT * FROM vista_empleados_it;
```

Consultar vistas existentes

```
SHOW FULL TABLES WHERE Table_type = 'VIEW';
```

Consultar la estructura de una vista

```
SHOW CREATE VIEW vista_empleados_it;

DESCRIBE vista_empleados_it;
```

Borrar una vista

```
DROP VIEW vista_empleados_it;
```

Ventajas de las vistas

- **Simplificación:** encapsulan consultas complejas en un nombre simple.
- **Seguridad:** permiten dar acceso solo a ciertas columnas/filas.
- **Mantenibilidad:** si la estructura cambia, solo se modifica la vista.
- **Reutilización:** se usan en múltiples consultas sin repetir código.

12. Resumen de palabras reservadas del temario

Categoría	Palabras clave
Selección	SELECT, FROM, AS, DISTINCT, *
Filtros	WHERE, AND, OR, NOT, IN, BETWEEN, LIKE, IS NULL, IS NOT NULL
Ordenación/Limitación	ORDER BY, ASC, DESC, LIMIT
Agregación	COUNT, SUM, AVG, MIN, MAX
Agrupación	GROUP BY, HAVING
Funciones lógicas	IF, IFNULL
Funciones numéricas	COUNT, ROUND, SUM, MIN, MAX, AVG
Funciones texto	CONCAT, UPPER, LOWER, LTRIM, RTRIM
Funciones fecha	NOW, CURRENT_TIMESTAMP, CURDATE, DATEDIFF, TIMEDIFF, YEAR, MONTH, DATE_FORMAT
JOINS	INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN, CROSS JOIN, NATURAL JOIN, ON
Conjuntos	UNION, INTERSECT, EXCEPT (MINUS)
Subconsultas	IN (subquery), EXISTS, NOT EXISTS, ALL, ANY
Vistas	CREATE VIEW, DROP VIEW, SHOW CREATE VIEW, DESCRIBE
Tablas derivadas	SELECT ... FROM (SELECT ...) AS alias